

ASSESSMENT TOOL - 1

COURSE CODE:MLA0201

NAME:B.HarshaVardhan

COURSE NAME:MACHINE LEARNING

Reg no. :192425201

1.Implement the concept of probability distribution to predict the likelihood of spam emails in a mail free trail system .

CODE:

```
import re, math
from collections import defaultdict

# Training data: (email text, label)
data = [
    ("win money now", "spam"),
    ("free lottery prize", "spam"),
    ("free trial offer now", "spam"),
    ("meeting tomorrow at office", "ham"),
    ("please review the report", "ham"),
    ("lunch schedule for tomorrow", "ham"),
]

word_counts = {"spam": defaultdict(int), "ham": defaultdict(int)}
class_counts = {"spam": 0, "ham": 0}
vocab = set()

# Train: build word frequency distribution per class
for text, label in data:
    class_counts[label] += 1
    for word in re.findall(r"[a-z]+", text.lower()):
        word_counts[label][word] += 1
        vocab.add(word)

total_emails = sum(class_counts.values())

def predict(text):
    words = re.findall(r"[a-z]+", text.lower())
    scores = {}
    for label in class_counts:
        prior = class_counts[label] / total_emails
        log_prob = math.log(prior)
        total_words = sum(word_counts[label].values())
        for w in words:
            count = word_counts[label][w]
            prob = (count + 1) / (total_words + len(vocab)) # Laplace smoothing
            log_prob += math.log(prob)
        scores[label] = log_prob

    # normalize log scores to probabilities
    max_score = max(scores.values())
    exp_scores = {k: math.exp(v - max_score) for k, v in scores.items()}
    total = sum(exp_scores.values())
    return {k: v / total for k, v in exp_scores.items()}

# Test
```

```

tests = ["free trial win now", "meeting report tomorrow"]
for email in tests:
    p = predict(email)
    print(f"\nEmail: {email}")
    print(f"P(Spam)={p['spam']*100:.2f}% P(Ham)={p['ham']*100:.2f}%")
    print("Verdict:", "SPAM" if p['spam'] > p['ham'] else "NOT SPAM")

```

OUTPUT:

```

=====
Email: free trial win now
P(Spam)=97.92% P(Ham)=2.08%
Verdict: SPAM

Email: meeting report tomorrow
P(Spam)=9.24% P(Ham)=90.76%
Verdict: NOT SPAM
>>>

```

2.Develop the design steps of a learning system to build a machine learning solution for student performance prediction.

CODE:

```

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

X = np.array([
    [2, 60, 50],
    [4, 70, 55],
    [6, 80, 65],
    [8, 90, 75],
    [3, 65, 52],
    [7, 85, 70],
    [5, 75, 60],
    [9, 95, 80],
    [1, 50, 40],
    [10, 98, 85],
])

y = np.array([52, 58, 68, 78, 55, 74, 63, 84, 42, 90]) # final scores
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("=" * 45)
print("MODEL EVALUATION")
print("=" * 45)
print(f"Mean Squared Error: {mse:.2f}")
print(f"R2 Score: {r2:.2f}")

```

```

new_student = np.array([[6, 78, 62]]) # study_hours, attendance%, past_score
predicted_score = model.predict(new_student)
print("\n" + "=" * 45)
print("NEW STUDENT PREDICTION")
print("=" * 45)
print(f"Input -> Study Hours: 6, Attendance: 78%, Past Score: 62")
print(f"Predicted Final Score: {predicted_score[0]:.2f}")
status = "PASS" if predicted_score[0] >= 50 else "FAIL"
print(f"Predicted Status: {status}")

```

OUTPUT:

```

===== RES1
=====
MODEL EVALUATION
=====
Mean Squared Error: 6.35
R2 Score: 0.90

=====
NEW STUDENT PREDICTION
=====
Input -> Study Hours: 6, Attendance: 78%, Past Score: 62
Predicted Final Score: 66.86
Predicted Status: PASS
>>>

```

3. Model the weather forecasting system with its outcome and challenges random variables and probability theory .

CODE:

```

import numpy as np
states = ["Sunny", "Cloudy", "Rainy"]
transition_matrix = np.array([
    [0.6, 0.3, 0.1], # from Sunny
    [0.3, 0.4, 0.3], # from Cloudy
    [0.2, 0.3, 0.5], # from Rainy
])
def simulate_weather(start_state, days):
    current = states.index(start_state)
    sequence = [states[current]]

    for _ in range(days):
        probs = transition_matrix[current] # probability distribution for tomorrow
        current = np.random.choice(len(states), p=probs) # sample from distribution
        sequence.append(states[current])

    return sequence
def predict_distribution(start_state, days):
    vector = np.zeros(len(states))
    vector[states.index(start_state)] = 1.0 # certainty today

    for _ in range(days):
        vector = vector @ transition_matrix # propagate distribution forward
    return dict(zip(states, vector))

print("=" * 50)
print("WEATHER SIMULATION (Random Variable Realization)")

```

```

print("=" * 50)
sim = simulate_weather("Sunny", 7)
print("7-Day Simulated Forecast:", sim)

print("\n" + "=" * 50)
print("PROBABILITY DISTRIBUTION FORECAST")
print("=" * 50)
for day in [1, 3, 7, 14]:
    dist = predict_distribution("Sunny", day)
    print(f"\nDay {day} forecast (starting from Sunny):")
    for state, prob in dist.items():
        print(f" P({state}) = {prob*100:.2f}%")

```

OUTPUT:

```

===== RESTART: D:\MLA0302\ML exp 3 Lab.py =====
=====
WEATHER SIMULATION (Random Variable Realization)
=====
7-Day Simulated Forecast: ['Sunny', 'Sunny', 'Sunny', 'Sunny', 'Sunny', 'Sunny', 'Cloudy', 'Cloudy']
=====
PROBABILITY DISTRIBUTION FORECAST
=====
Day 1 forecast (starting from Sunny):
P(Sunny) = 60.00%
P(Cloudy) = 30.00%
P(Rainy) = 10.00%

Day 3 forecast (starting from Sunny):
P(Sunny) = 42.10%
P(Cloudy) = 33.30%
P(Rainy) = 24.60%

Day 7 forecast (starting from Sunny):
P(Sunny) = 38.97%
P(Cloudy) = 33.33%
P(Rainy) = 27.70%

Day 14 forecast (starting from Sunny):
P(Sunny) = 38.89%
P(Cloudy) = 33.33%
P(Rainy) = 27.78%
>>>

```

4. Apply Conditional probability concept to design a fraud detection model suspensive banking transactions .

CODE:

```

import math
from collections import defaultdict
training_data = [
    (1, 1, 1, 1, "fraud"),
    (1, 1, 0, 1, "fraud"),
    (1, 0, 1, 1, "fraud"),
    (0, 1, 1, 0, "fraud"),
    (1, 1, 1, 0, "fraud"),
    (0, 0, 0, 0, "legit"),
    (0, 0, 0, 1, "legit"),
    (1, 0, 0, 0, "legit"),
    (0, 1, 0, 0, "legit"),
    (0, 0, 1, 0, "legit"),
    (0, 0, 0, 0, "legit"),
    (1, 0, 0, 0, "legit"),
]

feature_names = ["amount_high", "location_mismatch", "odd_hour", "high_frequency"]
class_counts = defaultdict(int)

```

```

feature_counts = defaultdict(lambda: defaultdict(int)) # feature_counts[label][(feature_index, value)]

for record in training_data:
    *features, label = record
    class_counts[label] += 1
    for idx, val in enumerate(features):
        feature_counts[label][(idx, val)] += 1

total_records = len(training_data)

def prior(label):
    return class_counts[label] / total_records
def likelihood(idx, val, label):
    count = feature_counts[label][(idx, val)]
    total = class_counts[label]
    return (count + 1) / (total + 2) # +2 since each feature is binary (0/1)
def predict(transaction_features):
    scores = {}
    for label in class_counts:
        log_prob = math.log(prior(label))
        for idx, val in enumerate(transaction_features):
            log_prob += math.log(likelihood(idx, val, label))
        scores[label] = log_prob

    # Normalize log scores into a proper probability distribution
    max_log = max(scores.values())
    exp_scores = {k: math.exp(v - max_log) for k, v in scores.items()}
    total = sum(exp_scores.values())
    return {k: v / total for k, v in exp_scores.items()}

test_transactions = [
    (1, 1, 1, 1), # high amount, location mismatch, odd hour, high frequency
    (0, 0, 0, 0), # normal transaction
    (1, 1, 0, 0), # high amount + location mismatch only
    (0, 1, 1, 0), # location mismatch + odd hour
]

print("=" * 60)
print("FRAUD DETECTION - CONDITIONAL PROBABILITY (Bayes' Theorem)")
print("=" * 60)

THRESHOLD = 0.5

for txn in test_transactions:
    probs = predict(txn)
    fraud_prob = probs.get("fraud", 0) * 100
    legit_prob = probs.get("legit", 0) * 100
    verdict = "SUSPICIOUS / FLAGGED" if probs["fraud"] > THRESHOLD else "LEGITIMATE"

    print(f"\nTransaction: {dict(zip(feature_names, txn))}")
    print(f" P(Fraud) = {fraud_prob:.2f}%")
    print(f" P(Legit) = {legit_prob:.2f}%")
    print(f" Decision : {verdict}")

```

OUTPUT:

```
===== RESTART: D:\MLA0302\ML exp 4 Lab.py =====
=====
FRAUD DETECTION - CONDITIONAL PROBABILITY (Bayes' Theorem)
=====

Transaction: {'amount_high': 1, 'location_mismatch': 1, 'odd_hour': 1, 'high_frequency': 1}
P(Fraud) = 97.60%
P(Legit) = 2.40%
Decision : SUSPICIOUS / FLAGGED

Transaction: {'amount_high': 0, 'location_mismatch': 0, 'odd_hour': 0, 'high_frequency': 0}
P(Fraud) = 2.23%
P(Legit) = 97.77%
Decision : LEGITIMATE

Transaction: {'amount_high': 1, 'location_mismatch': 1, 'odd_hour': 0, 'high_frequency': 0}
P(Fraud) = 49.90%
P(Legit) = 50.10%
Decision : LEGITIMATE

Transaction: {'amount_high': 0, 'location_mismatch': 1, 'odd_hour': 1, 'high_frequency': 0}
P(Fraud) = 63.54%
P(Legit) = 36.46%
Decision : SUSPICIOUS / FLAGGED
>>> |
```

5.illustrate how biases theory can be applied in medical Diagnosis for disease prediction.

CODE:

```
def bayes_diagnosis(prevalence, sensitivity, specificity):
    p_disease = prevalence
    p_no_disease = 1 - prevalence
    p_pos_given_disease = sensitivity
    p_pos_given_no_disease = 1 - specificity # false positive rate
    p_positive = (p_pos_given_disease * p_disease) + (p_pos_given_no_disease * p_no_disease)
    p_disease_given_positive = (p_pos_given_disease * p_disease) / p_positive
    return p_disease_given_positive, p_positive
print("=" * 60)
print("MEDICAL DIAGNOSIS - BAYES' THEOREM (Single Test)")
print("=" * 60)
```

```
prevalence = 0.01    # 1% of population has the disease
sensitivity = 0.99   # test correctly detects disease 99% of the time
specificity = 0.95   # test correctly rules out disease 95% of the time
```

```
posterior, p_pos = bayes_diagnosis(prevalence, sensitivity, specificity)
```

```
print(f"\nDisease Prevalence    : {prevalence*100:.2f}%")
print(f"Test Sensitivity       : {sensitivity*100:.2f}%")
print(f"Test Specificity        : {specificity*100:.2f}%")
print(f"P(Positive Test overall): {p_pos*100:.2f}%")
print(f"\n=> P(Disease | Positive Test) = {posterior*100:.2f}%")
print("(Even with a positive result, true disease probability is much")
print("lower than 99% because the disease is rare — false positives dominate)")
import math
from collections import defaultdict
training_data = [
    (1, 1, 1, 1, "disease"),
    (1, 1, 1, 0, "disease"),
    (1, 0, 1, 1, "disease"),
    (0, 1, 1, 1, "disease"),
    (1, 1, 0, 1, "disease"),
    (0, 0, 0, 0, "healthy"),
    (1, 0, 0, 0, "healthy"),
```

```

(0, 1, 0, 0, "healthy"),
(0, 0, 1, 0, "healthy"),
(1, 0, 0, 0, "healthy"),
(0, 0, 0, 0, "healthy"),
]

symptom_names = ["fever", "cough", "fatigue", "breathlessness"]

class_counts = defaultdict(int)
symptom_counts = defaultdict(lambda: defaultdict(int))

for record in training_data:
    *symptoms, label = record
    class_counts[label] += 1
    for idx, val in enumerate(symptoms):
        symptom_counts[label][(idx, val)] += 1

total_records = len(training_data)

def prior(label):
    return class_counts[label] / total_records

def likelihood(idx, val, label):
    count = symptom_counts[label][(idx, val)]
    total = class_counts[label]
    return (count + 1) / (total + 2) # Laplace smoothing, binary feature

def predict_disease(symptom_vector):
    scores = {}
    for label in class_counts:
        log_prob = math.log(prior(label))
        for idx, val in enumerate(symptom_vector):
            log_prob += math.log(likelihood(idx, val, label))
        scores[label] = log_prob

    max_log = max(scores.values())
    exp_scores = {k: math.exp(v - max_log) for k, v in scores.items()}
    total = sum(exp_scores.values())
    return {k: v / total for k, v in exp_scores.items()}

print("\n" + "=" * 60)
print("MULTI-SYMPTOM NAIVE BAYES DIAGNOSIS")
print("=" * 60)

patients = [
    (1, 1, 1, 1), # all symptoms present
    (0, 0, 0, 0), # no symptoms
    (1, 0, 1, 0), # fever + fatigue only
]

for symptoms in patients:
    probs = predict_disease(symptoms)
    disease_prob = probs.get("disease", 0) * 100
    healthy_prob = probs.get("healthy", 0) * 100
    verdict = "LIKELY DISEASE" if probs["disease"] > 0.5 else "LIKELY HEALTHY"

```

```

print(f"\nSymptoms: {dict(zip(symptom_names, symptoms))}")
print(f" P(Disease) = {disease_prob:.2f}%")
print(f" P(Healthy) = {healthy_prob:.2f}%")
print(f" Diagnosis : {verdict}")

```

OUTPUT:

```

===== RESTART: D:\MLA0302\ML exp 5 Lab
=====
MEDICAL DIAGNOSIS - BAYES' THEOREM (Single Test)
=====

Disease Prevalence      : 1.00%
Test Sensitivity        : 99.00%
Test Specificity        : 95.00%
P(Positive Test overall): 5.94%

=> P(Disease | Positive Test) = 16.67%
(Even with a positive result, true disease probability is much
lower than 99% because the disease is rare - false positives dominate)

=====
MULTI-SYMPOM NAIVE BAYES DIAGNOSIS
=====

Symptoms: {'fever': 1, 'cough': 1, 'fatigue': 1, 'breathlessness': 1}
P(Disease) = 98.67%
P(Healthy) = 1.33%
Diagnosis : LIKELY DISEASE

Symptoms: {'fever': 0, 'cough': 0, 'fatigue': 0, 'breathlessness': 0}
P(Disease) = 1.77%
P(Healthy) = 98.23%
Diagnosis : LIKELY HEALTHY

Symptoms: {'fever': 1, 'cough': 0, 'fatigue': 1, 'breathlessness': 0}
P(Disease) = 36.07%
P(Healthy) = 63.93%
Diagnosis : LIKELY HEALTHY
>>> |

```

6. Construct machine learning workflow to classify handwritten design by apolysis the system of a learning system.

CODE:

```

from sklearn.datasets import load_digits

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

digits = load_digits()

X = digits.data    # 1797 samples, 64 features (8x8 pixel values flattened)

y = digits.target  # labels: 0-9

print("=" * 55)

print("DATASET OVERVIEW")

print("=" * 55)

print(f"Total samples : {X.shape[0]}")

```



```

print(f'Features/image: {X.shape[1]} (8x8 pixels flattened)')

print(f'Classes      : {set(y)}')

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

model = SVC(kernel="rbf", gamma="scale")

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("\n" + "=" * 55)

print("MODEL EVALUATION")

print("=" * 55)

print(f'Accuracy: {accuracy*100:.2f}%')

print("\nConfusion Matrix:")

print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")

print(classification_report(y_test, y_pred))

sample_index = 5

sample = X_test[sample_index].reshape(1, -1)

predicted_digit = model.predict(sample)[0]

actual_digit = y_test[sample_index]

print("=" * 55)

print("SAMPLE PREDICTION")

```

```

print("=" * 55)

print(f'Predicted Digit: {predicted_digit}')

print(f'Actual Digit : {actual_digit}')-

import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 5, figsize=(10, 3))

for i, ax in enumerate(axes):

    img = X_test[i].reshape(8, 8)

    ax.imshow(img, cmap="gray")

    ax.set_title(f'Pred: {y_pred[i]}')

    ax.axis("off")

plt.tight_layout()

plt.show()

```

OUTPUT:

